

Technical Report

MechanicPro

Management System for a Mechanic Shop

System Architecture, Database Design, and Technical Overview

Author Eduardo Freitas Fernandes

Date June 2026

Document Technical Report

Version 1.0.0

Status Final

Language English

Abstract

This document is the technical report for **MechanicPro**, a desktop application designed to manage the day-to-day operations of a mechanic shop. The report describes the system architecture, the technology stack, the database design, and the build and release process.

MechanicPro is built with **Tauri v2** and **Rust** for the desktop shell, **React 19** and **TypeScript** for the user interface, and **SQLite** for local data persistence. The application runs entirely offline, with no server or cloud dependency.

The intended reader is a developer who wants to understand, maintain, or extend the system. Usage instructions are covered separately in the User Manual.

Keywords: Tauri, React, TypeScript, SQLite, Desktop Application, Mechanic Shop Management.

Revision History

Version	Date	Description	Author
1.0.0	June 2026	Initial document created	Eduardo Fernandes

This document describes version 1.0.0 of MechanicPro. For changes to the application itself, refer to the project `CHANGELOG.md`.

Contents

1	Introduction	6
1.1	Purpose	6
1.2	Scope	6
1.3	Intended Audience	6
1.4	Document Conventions	7
2	System Overview	8
2.1	Application Purpose	8
2.2	Technology Stack	8
2.3	System Requirements	9
2.4	High-Level Data Flow	9
3	Architecture	11
3.1	Overview	11
3.2	Frontend - React and TypeScript	11
3.2.1	Component Structure	11
3.2.2	State Management	12
3.2.3	Routing	12
3.3	Backend - Tauri and Rust	13
3.3.1	Tauri Commands	13
3.3.2	Plugin Usage	13
3.4	IPC Communication	13
3.5	Folder Structure	14
4	Database	15
4.1	Overview	15
4.2	Schema	15
4.2.1	clients	15
4.2.2	vehicles	16
4.2.3	service_orders	16
4.2.4	service_operations	17
4.3	Entity-Relationship Diagram	18
4.4	Business Rules	18
4.5	CSV Format	19
5	Build and Release	21
5.1	Development Environment Setup	21
5.2	Build Process	21
5.3	Output Artefacts	22
5.4	Configuration	22
6	Known Limitations	24
A	Glossary	25
B	References	26

List of Figures

- 2.1 High-level data flow from user interaction to SQLite and back 10
- 4.1 Entity-Relationship Diagram for the MechanicPro database 18

List of Tables

- 2.1 MechanicPro technology stack 8
- 3.1 Reusable components in `src/components/` 12
- 3.2 Application routes 13
- 4.1 Schema for the `clients` table 16
- 4.2 Schema for the `vehicles` table 16
- 4.3 Schema for the `service_orders` table 17
- 4.4 Schema for the `service_operations` table 17
- 4.5 Clients CSV column specification 19
- 4.6 Vehicles CSV column specification 19
- 4.7 Services CSV column specification 20
- 5.1 Build output artefacts by platform 22
- 5.2 Key `tauri.conf.json` settings 23

1 Introduction

1.1 Purpose

This document is the technical reference for **MechanicPro** version 1.0.0. Its purpose is to describe the internal design of the system in sufficient detail that a developer can understand, maintain, or extend the application without prior knowledge of the codebase.

The report covers the system architecture, the technology stack, the database schema, the state management strategy, and the build and release process. It does *not* cover end-user workflows or feature usage; those topics are addressed separately in the User Manual.

1.2 Scope

This document applies exclusively to the **desktop application** component of MechanicPro. The following topics are in scope:

- Overall system architecture and the two-process model (WebView + Tauri shell).
- Frontend structure: pages, components, routing, and state management.
- Backend integration: the Tauri SQL plugin and the service layer.
- Database design: schema, relationships, business rules, and soft-delete strategy.
- CSV import and export format specification.
- Development environment setup, production build process, and output artefacts.
- Known technical limitations of the current release.

The following topics are explicitly **out of scope**: user interface usage instructions, network infrastructure (the application has none), and future roadmap items.

1.3 Intended Audience

This document is intended for **developers** who need to work with the MechanicPro codebase. The reader is assumed to have working knowledge of:

- TypeScript and the React component model.
- Relational database concepts (tables, primary keys, foreign keys).
- Basic command-line usage (running `npm` and `cargo` commands).

No prior knowledge of Tauri or Rust is required to understand the frontend sections of this report. The backend sections that touch on Tauri configuration are self-contained and include sufficient context.

1.4 Document Conventions

The following conventions are used throughout this report:

- **Bold text** is used to introduce key terms or to emphasise important statements.
- **Coloured bold text** (e.g., **Zustand**) denotes a named technology, library, or architectural concept.
- `Monospace text` is used for file names, directory paths, command-line instructions, and inline code references.
- Code blocks with line numbers are used for multi-line source code and shell commands.
- Blue information boxes are used to call attention to notable constraints, warnings, or design decisions.

Note: Blue boxes like this one highlight important information or design decisions that deserve special attention.

2 System Overview

2.1 Application Purpose

MechanicPro is a desktop application designed to manage the day-to-day operations of a small mechanic shop. It provides a centralised, offline system for recording and querying:

- **Clients** — the vehicle owners who bring their cars for service.
- **Vehicles** — each registered car, associated with one client.
- **Service Orders** — repair and maintenance records linked to a vehicle, including itemised operations, mileage, labour hours, and total price.

The application also provides a **Dashboard** with summary statistics and trend charts, and supports **CSV import and export** to allow data migration and backup. Service orders can be printed or downloaded as a **PDF invoice**.

MechanicPro runs entirely **offline**. It has no server, no cloud dependency, and no network requirement at runtime. All data is stored locally in a single SQLite database file on the user’s machine.

2.2 Technology Stack

Table 2.1 lists the libraries and frameworks that make up the MechanicPro technology stack.

Layer	Technology	Role
UI Framework	React 19 + TypeScript	Component-based frontend interface
Build Tool	Vite 7	Development server and production bundler
Desktop Shell	Tauri v2 (Rust)	Native window, OS integration, and IPC bridge
Database	SQLite via plugin-sql	Local, file-based data persistence
State Management	Zustand 5	Global client-side state and mutation orchestration
Routing	react-router-dom v7	Client-side page navigation
Charts	recharts	Dashboard statistics visualisations
PDF Generation	jsPDF + html2canvas	Service order invoice export
Icons	lucide-react	Consistent UI icon set

Table 2.1: MechanicPro technology stack

2.3 System Requirements

Runtime Requirements

- **Windows 10 / 11** (x64): Microsoft **WebView2** runtime must be installed. It is bundled automatically in the distributed installer.
- **macOS 10.15 Catalina or later**: Uses the system **WKWebView**. No additional runtime is required.
- **Linux** (x64): A compatible **WebKitGTK** installation is required (typically available by default on modern desktop distributions).

Hardware Requirements

MechanicPro has no demanding hardware requirements. Any machine capable of running a modern web browser is sufficient. Approximate minimum specifications:

- **RAM**: 256 MB available (the application is lightweight).
- **Disk**: 50 MB for the installer; additional space for the SQLite database, which grows proportionally to the volume of records.
- **Display**: 1024 × 768 minimum resolution. The UI is optimised for 1280 × 800 and above.

Development Prerequisites

Building from source additionally requires:

- **Node.js** 18 or later (with `npm`).
- **Rust** stable toolchain (via `rustup`).
- Platform-specific Tauri build dependencies as documented at <https://tauri.app/start/prerequisites/>.

2.4 High-Level Data Flow

All data access in MechanicPro follows a strict, layered path from the UI down to the SQLite database and back. Figure 2.1 illustrates this flow at a high level.

1. **User Interaction**: The user performs an action in the React UI (e.g., clicks “Save” on a new client form).
2. **Zustand Store**: The UI calls an action on the relevant **Zustand** store (e.g., `useClientStore.createClient()`). The store sets a loading flag and delegates to the service layer.
3. **Service Layer**: A TypeScript service module (e.g., `clientService.ts`) constructs the SQL query and calls `db.execute()` or `db.select()`.
4. **Tauri SQL Plugin**: The `@tauri-apps/plugin-sql` library serialises the query and parameters as JSON and sends them over the **IPC** bridge to the Tauri Rust core.
5. **Rust Core / SQLite**: The Rust side of the plugin executes the query against the local SQLite file (`mechanicpro.db`) and returns the result as a JSON payload.

6. **Store Update:** The Zustand store receives the result, updates its state (e.g., the `clients` array), and clears the `loading` flag. React re-renders only the components that subscribe to the changed state slice.

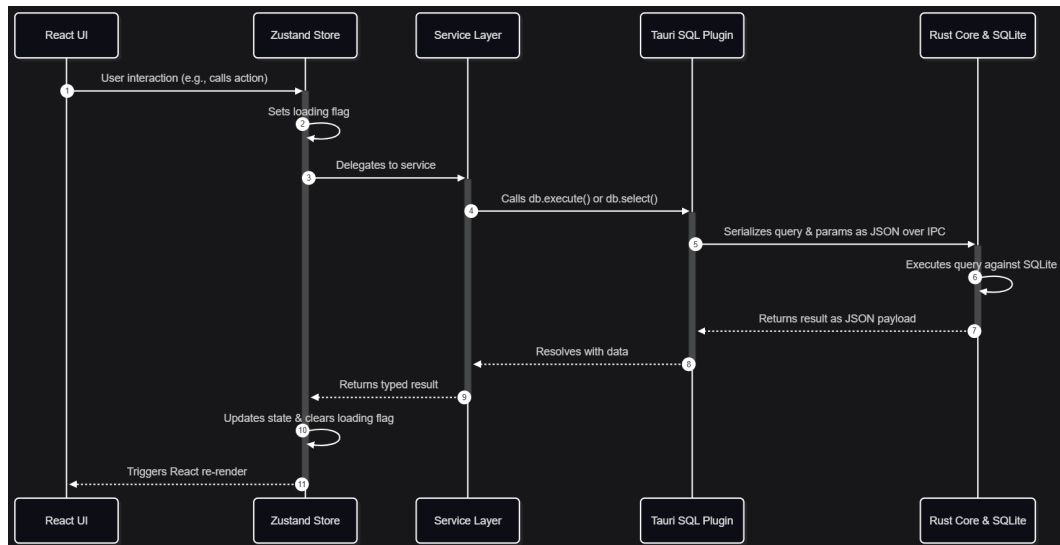


Figure 2.1: High-level data flow from user interaction to SQLite and back

Note: There are no custom Rust commands in MechanicPro. All database access is handled entirely from the TypeScript side using the **Tauri SQL plugin's** JavaScript bindings. The Rust layer is used solely for its native window, OS integration, and IPC capabilities.

3 Architecture

3.1 Overview

MechanicPro follows the [Tauri two-process model](#). The application is composed of two isolated processes that communicate exclusively through a structured IPC bridge:

- **The WebView process** — a Chromium-based (Windows) or WebKit-based (macOS/Linux) renderer that runs the React + TypeScript frontend. This process has no direct access to the filesystem, operating system APIs, or the SQLite file.
- **The Rust core process** — the native Tauri host process, compiled from Rust. It owns the application window, manages OS permissions, and hosts the plugin runtime that has privileged access to system resources including the SQLite database.

The two processes cannot share memory. All data exchange between the frontend and the Rust core is serialised to JSON and passed through Tauri's [IPC bridge](#). In MechanicPro, this bridge is used exclusively by the `@tauri-apps/plugin-sql` library — there are no hand-authored Tauri commands or Rust source files in the project beyond the default Tauri scaffold.

3.2 Frontend - React and TypeScript

3.2.1 Component Structure

The frontend source lives in `app/src/` and is divided into four categories:

- **Pages** (`src/pages/`): Route-level components, each corresponding to a distinct view in the application. There are eight pages in total (see [Section 3.2.3](#)).
- **Components** (`src/components/`): Reusable UI elements shared across pages. These include modal dialogs and structural components.
- **Stores** (`src/store/`): Zustand global state modules (see [Section 3.2.2](#)).
- **Services** (`src/services/`): TypeScript modules that encapsulate all SQL queries and business logic (see [Section 3.4](#)).

Table [3.1](#) lists the components in `src/components/` and their roles.

Component	Role
Layout.tsx	Root shell; renders the sidebar and the <Outlet /> for nested routes
Sidebar.tsx	Navigation menu with links to all pages
ClientModal.tsx	Modal form for creating and editing clients
VehicleModal.tsx	Modal form for creating and editing vehicles
ConfirmModal.tsx	Generic confirmation dialog for destructive actions
VehicleHistoryOverlay.tsx	Slide-in panel displaying the full service history for a vehicle
PrintTemplate.tsx	Hidden DOM element used by html2canvas for PDF generation

Table 3.1: Reusable components in `src/components/`

3.2.2 State Management

Global application state is managed with **Zustand** (v5). Three stores are defined in `src/store/`, one per primary entity domain:

- `useClientStore.ts` — holds the `clients` array, the active search string, and async mutation actions (`createClient`, `updateClient`, `deleteClient`).
- `useVehicleStore.ts` — holds the `vehicles` array, the active search string, and the equivalent mutation actions.
- `useServiceOrderStore.ts` — holds the `serviceOrders` array, the active search string, the sort order preference (ASC/DESC), and mutation actions.

Each store follows the same pattern: every mutation action (`create`, `update`, `delete`) calls the corresponding service function, then automatically calls `fetchX()` to refresh the list. This ensures that all components subscribed to the store see the updated data without requiring manual callback chains or prop drilling.

Search terms and sort preferences are persisted to `sessionStorage` so that navigation between pages does not reset the user's last query.

Design Note: Zustand stores act as both the state container and the hook interface. Components call `useClientStore()` directly; no additional custom hook wrapper layer is needed.

3.2.3 Routing

Client-side routing is handled by **react-router-dom** v7. The route tree is defined in `src/App.tsx` using a single nested layout route:

Path	Component	Description
/	DashboardPage	Summary statistics and trend charts
/clients	ClientsPage	Searchable client list with CRUD
/vehicles	VehiclesPage	Searchable vehicle list with CRUD
/services	ServicesPage	Service order list with search and sort
/services/new	CreateServiceOrderPage	Multi-step form to create a service order
/services/:id	ServiceOrderDetailsPage	Detail view, print, and PDF export
/import	ImportPage	CSV data import for all three entities
/export	ExportPage	CSV data export for all three entities

Table 3.2: Application routes

All routes are rendered inside the `Layout` component, which provides the persistent sidebar navigation. There is no lazy loading; all page components are bundled into a single chunk.

3.3 Backend - Tauri and Rust

3.3.1 Tauri Commands

MechanicPro does **not** define any custom Tauri commands. The default Tauri scaffold generates a minimal `src-tauri/src/main.rs` that registers the application and the required plugins. No additional `#[tauri::command]` functions have been authored.

All privileged operations (database access) are delegated to the `plugin-sql` plugin, which exposes its own pre-built command surface over the IPC bridge.

3.3.2 Plugin Usage

A single Tauri plugin is used in production:

- `@tauri-apps/plugin-sql` (JS) / `tauri-plugin-sql` (Rust): Provides `db.execute()` and `db.select()` methods in the TypeScript layer, which are transmitted over IPC to the Rust side and executed against a local SQLite file. The plugin is declared in `tauri.conf.json` under the `plugins.sql` key and initialised on first call to `Database.load("sqlite:mechanicpro.db")`.

3.4 IPC Communication

The **IPC bridge** in Tauri transmits messages as JSON over a local channel between the `WebView` and the Rust process. In MechanicPro, IPC calls are never made directly by application code. Instead, all IPC is mediated through the service layer:

1. A service function calls `db.execute(sql, params)` or `db.select<T[]>(sql, params)`.
2. The plugin serialises the query string and the parameter array to JSON and dispatches a `Tauri invoke()` call internally.
3. The Rust plugin handler deserialises the payload, binds the parameters, executes the query, and serialises the row results back to JSON.

4. The TypeScript Promise resolves with the typed result.

SQL parameters are always passed as a positional array (using ? placeholders), which prevents SQL injection. Errors thrown on the Rust side are propagated as rejected Promises and caught by the calling store action, which stores the error message in its error state field.

3.5 Folder Structure

The annotated repository layout is shown in Listing 3.1.

Listing 3.1: Repository structure

```

1 MechanicPro/
2 +--- app/                                # Vite + React + Tauri
    |   frontend project
3   |   +--- src/
4   |   |   +--- components/              # Reusable UI components
5   |   |   +--- pages/                   # Route-level page components
6   |   |   +--- services/                # SQL query and business logic
    |   modules
7   |   |   +--- store/                   # Zustand global state stores
8   |   |   +--- db.ts                    # DB singleton, table init,
    |   TypeScript interfaces
9   |   |   +--- App.tsx                  # Router definition
10  |   |   +--- App.css                   # Global design system styles
11  |   |   '--- main.tsx                  # React entry point
12  |   +--- src-tauri/
13  |   |   +--- src/
14  |   |   |   '--- main.rs               # Tauri app entry point and
    |   plugin registration
15  |   |   +--- Cargo.toml                # Rust dependencies
16  |   |   '--- tauri.conf.json           # Tauri configuration (app name,
    |   version, plugins)
17  |   +--- package.json
18  |   '--- vite.config.ts
19 +--- docs/                              # Technical documentation
20 |   +--- figures/                        # Images referenced in the
    |   report
21 |   '--- TechnicalReport.tex            # This document
22 +--- CHANGELOG.md
23 '--- README.md

```

4 Database

4.1 Overview

MechanicPro uses **SQLite** as its sole data store. The database is a single file named `mechanicpro.db`, managed by the `tauri-plugin-sql` Rust plugin. The file is created automatically on first launch and stored in the platform's standard application data directory:

- **Windows:** `%APPDATA%\MechanicPro\mechanicpro.db`
- **macOS:** `~/Library/Application Support/MechanicPro/mechanicpro.db`
- **Linux:** `~/.local/share/MechanicPro/mechanicpro.db`

The schema is initialised idempotently on every application start via `CREATE TABLE IF NOT EXISTS` and `CREATE UNIQUE INDEX IF NOT EXISTS` statements executed in `src/db.ts`. No migration framework is used; the schema is designed to be stable across the v1.0.0 lifecycle.

Note: The SQLite database file is stored in plaintext. No encryption at rest is applied in this version. For sensitive deployments, filesystem-level encryption should be considered.

4.2 Schema

The database consists of four tables. All tables implement a **soft-delete** pattern: records are never physically removed. Instead, a `deleted_at` timestamp is set, and all queries filter on `deleted_at IS NULL` to exclude deleted rows.

4.2.1 `clients`

Stores the vehicle owners registered in the system.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	Unique client identifier
name	TEXT	NOT NULL	Full name of the client
phone	TEXT	—	Optional contact phone number
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
deleted_at	DATETIME	—	Soft-delete timestamp; NULL means active

Table 4.1: Schema for the `clients` table

A partial unique index on `name` (where `deleted_at IS NULL`) enforces that no two *active* clients share the same name. Deleted records are excluded from the uniqueness constraint, so a name can be re-registered after the original record is deleted.

4.2.2 vehicles

Stores vehicles, each belonging to exactly one client.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	Unique vehicle identifier
client_id	INTEGER	NOT NULL, FK → <code>clients.id</code>	Owner of the vehicle
plate	TEXT	NOT NULL	Licence plate in uppercase
brand	TEXT	NOT NULL	Vehicle manufacturer (e.g., Toyota)
model	TEXT	NOT NULL	Vehicle model name
year	INTEGER	—	Optional model year
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
deleted_at	DATETIME	—	Soft-delete timestamp; NULL means active

Table 4.2: Schema for the `vehicles` table

A partial unique index on `plate` (where `deleted_at IS NULL`) prevents duplicate licence plates among active vehicles. Plates are normalised to uppercase by the service layer before insertion.

4.2.3 service_orders

The central entity of the application. Each record represents one repair or maintenance visit for a vehicle.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	Unique order identifier
vehicle_id	INTEGER	NOT NULL, FK → vehicles.id	Vehicle serviced
mileage	INTEGER	NOT NULL	Vehicle odometer reading at time of service
hours	REAL	NOT NULL	Labour hours worked
hourly_rate	REAL	NOT NULL	Labour rate per hour
observations	TEXT	—	Optional free-text notes
total_price	REAL	NOT NULL	Computed total (labour + operations); stored at insert time
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time of the service order
deleted_at	DATETIME	—	Soft-delete timestamp; NULL means active

Table 4.3: Schema for the `service_orders` table

4.2.4 `service_operations`

Stores the individual line items (parts replaced, repairs performed) that belong to a service order. There is no soft delete on this table; operations are physically deleted when the parent order is soft-deleted (cascade is handled at the application layer).

Column	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	Unique operation identifier
service_order_id	INTEGER	NOT NULL, FK → service_orders.id	Parent service order
description	TEXT	NOT NULL	Description of the operation or part
price	REAL	NOT NULL	Price charged for this line item

Table 4.4: Schema for the `service_operations` table

4.3 Entity-Relationship Diagram

Figure 4.1 illustrates the relationships between the four tables.

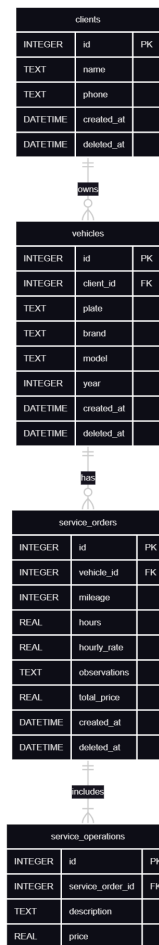


Figure 4.1: Entity-Relationship Diagram for the MechanicPro database

4.4 Business Rules

The following business rules are enforced at the application and database level:

- **Client name uniqueness:** A partial unique index on `clients(name)` WHERE `deleted_at` IS NULL ensures no two active clients share a name. The check is also performed application-side in `clientService.ts` to provide a user-friendly error message before the constraint fires.
- **Licence plate uniqueness:** Equivalent partial unique index on `vehicles(plate)` WHERE `deleted_at` IS NULL. Plates are normalised to uppercase before comparison.
- **Total price calculation:** The `total_price` of a service order is calculated at insert time as:

$$\text{total_price} = (\text{hours} \times \text{hourly_rate}) + \sum_{\text{operations}} \text{price}$$

The value is stored in the database and not recalculated on read, ensuring historical records remain accurate even if rates change.

- **Soft deletes:** No DELETE statements are issued against the three main tables. All queries that list data filter on `deleted_at IS NULL`.
- **Required fields:** `clients.name`, `vehicles.client_id`, `vehicles.plate`, `vehicles.brand`, `vehicles.model`, `service_orders.vehicle_id`, `service_orders.mileage`, `service_orders.hours`, `service_orders.hourly_rate`, and `service_operations.description` are all NOT NULL.

4.5 CSV Format

All CSV files use **comma** as the delimiter and **UTF-8** encoding with a BOM (U+FEFF) prepended for Microsoft Excel compatibility. Values that contain commas, double quotes, or newlines are wrapped in double quotes and internal quotes are escaped as `"`.

Clients CSV (`clientes_{date}.csv`)

Header	Type	Notes
ID	Integer	Auto-generated database identifier
Nome	String	Client name; required on import
Telefone	String	Optional; empty string if not set
Data de Criação	String	Date formatted as dd/mm/yyyy (pt-PT locale)

Table 4.5: Clients CSV column specification

Vehicles CSV (`veiculos_{date}.csv`)

Header	Type	Notes
ID	Integer	Auto-generated database identifier
Matrícula	String	Licence plate in uppercase; required; must be unique
Marca	String	Brand name; required
Modelo	String	Model name; required
Ano	Integer	Optional; empty string if not set
Cliente	String	Must exactly match an existing active client's name
Data de Criação	String	Date formatted as dd/mm/yyyy (pt-PT locale)

Table 4.6: Vehicles CSV column specification

Services CSV (servicos_{date}.csv)

Header	Type	Notes
ID	Integer	Auto-generated database identifier
Cliente	String	Client name
Matrícula	String	Licence plate; must match an existing active vehicle
Quilometragem	Integer	Odometer reading in kilometres
Horas	Decimal	Labour hours worked
Preço/Hora	Decimal	Hourly labour rate
Operações: Preço	String	Pipe-separated list of description: price entries
Observações	String	Optional free-text notes
Data	String	Date formatted as dd/mm/yyyy (pt-PT locale)

Table 4.7: Services CSV column specification

Import ordering dependency: CSV files must be imported in the following order: **Clients** first, then **Vehicles**, then **Services**. Each entity references the previous one by name or plate, and the import will skip rows where the referenced entity is not

5 Build and Release

5.1 Development Environment Setup

The following prerequisites must be installed before working with the MechanicPro source code:

- **Node.js** 18 or later with npm.
- **Rust** stable toolchain, installed via rustup (<https://rustup.rs>).
- **Platform-specific Tauri dependencies** documented at <https://tauri.app/start/prerequisites/> (e.g., WebView2 SDK on Windows, webkit2gtk on Linux).

Once the prerequisites are in place, clone the repository and run the following commands from the `app/` directory:

Listing 5.1: Install dependencies and start the development server

```
1 cd app
2 npm install
3 npm run tauri dev
```

This starts the Vite development server on `http://localhost:1420` and launches a Tauri window pointed at that URL. Hot module replacement (HMR) is active for the React frontend; Rust changes require a full restart.

5.2 Build Process

The production build is triggered with a single command from the `app/` directory:

Listing 5.2: Production build command

```
1 npm run tauri build
```

This command executes the following pipeline in sequence:

1. **Frontend build** (`npm run build`): Vite compiles and bundles all TypeScript and CSS into optimised static assets in `app/dist/`.
2. **Rust compilation** (`cargo build -release`): The Tauri Rust crate is compiled with the `release` profile. This step embeds the frontend `dist/` assets directly into the binary.
3. **Packaging**: Tauri wraps the compiled binary into a platform-native installer or bundle according to the `bundle.targets` setting in `tauri.conf.json`.

Note: The first `tauri` build on a new machine may take several minutes as Cargo downloads and compiles all Rust dependencies. Subsequent builds are significantly faster due to incremental compilation.

5.3 Output Artefacts

Build outputs are placed in `app/src-tauri/target/release/bundle/`. The artefacts produced depend on the host platform:

Platform	Format	Notes
Windows	<code>.msi</code>	Windows Installer package; bundles WebView2 runtime
Windows	<code>.exe (NSIS)</code>	Alternative standalone installer
macOS	<code>.dmg</code>	Disk image containing the <code>.app</code> bundle
macOS	<code>.app</code>	macOS application bundle
Linux	<code>.deb</code>	Debian/Ubuntu package
Linux	<code>.AppImage</code>	Portable, distribution-independent binary

Table 5.1: Build output artefacts by platform

The `bundle.targets` key in `tauri.conf.json` is set to `"all"`, meaning all supported formats for the host platform are produced in a single build run.

5.4 Configuration

Key application settings are centralised in `app/src-tauri/tauri.conf.json`. Table 5.2 documents the most relevant fields for a developer maintaining or releasing the application.

Key	Value	Purpose
productName	MechanicPro	Display name used in installers and window title
version	1.0.0	Application version embedded in the installer and About dialog
identifier	com.asus.mechanicpro	Reverse-domain bundle identifier; used by the OS for app data directories
devUrl	http://localhost:1420	URL loaded by Tauri during tauri dev
frontendDist	../dist	Path to the Vite build output, embedded into the binary
windows[0].width	800	Initial and minimum window width in pixels
windows[0].height	600	Initial and minimum window height in pixels
windows[0].maximized	true	Window launches maximised by default
bundle.targets	"all"	Produce all platform-appropriate installer formats

Table 5.2: Key `tauri.conf.json` settings

Release checklist: Before building a release, ensure that (1) `version` in `tauri.conf.json` is updated, (2) `version` in `app/package.json` matches, (3) `CHANGELOG.md` is updated, and (4) the build passes `tsc` without errors.

6 Known Limitations

This chapter documents the known technical limitations of MechanicPro v1.0.0. These are not bugs; they are deliberate trade-offs made to keep the application simple and maintainable for a single-user, single-machine context.

- **Single-user only.** The SQLite database file is accessed exclusively by the local application instance. There is no concurrency control, no shared network database, and no multi-user or multi-device support.
- **No data encryption at rest.** The `mechanicpro.db` file is a standard, unencrypted SQLite file. Any user with access to the filesystem can open it with any SQLite client. For deployments handling sensitive data, filesystem-level encryption (e.g., BitLocker, FileVault) is recommended.
- **No automatic backup mechanism.** The application does not perform automated backups of the database file. The recommended backup strategy is to periodically copy `mechanicpro.db` to an external location or use the CSV export feature.
- **No authentication or access control.** Any user who can launch the application has full read and write access to all data. There is no login screen, no role-based permissions, and no audit log.
- **No network or cloud synchronisation.** The application is entirely offline. There is no API, no sync engine, and no ability to share data between multiple installations automatically.
- **No database migration framework.** Schema changes between versions require manual migration or a full data re-import. The v1.0.0 schema is designed to be final for this release cycle.
- **PDF generation limitations.** Invoice PDFs are generated using [html2canvas](#) to render an off-screen DOM element. Complex CSS layouts or non-Latin characters may not render correctly in all cases. The output is rasterised, not vector-based.
- **Import does not update existing records.** The CSV import feature is append-only. If a client or vehicle already exists with the same name or plate, the row is silently skipped. There is no “upsert” behaviour.

A Glossary

CRUD	Create, Read, Update, Delete - the four basic operations on persistent data.
IPC	Inter-Process Communication. In Tauri, the mechanism by which the WebView (frontend) sends messages to the Rust core (backend) and receives responses.
Soft Delete	A deletion strategy where records are not physically removed from the database. Instead, a <code>deleted_at</code> timestamp is set. Queries exclude soft-deleted rows by filtering on <code>deleted_at IS NULL</code> .
SQLite	A self-contained, serverless, file-based relational database engine. The entire database is stored in a single <code>.db</code> file on disk.
Tauri	A framework for building desktop applications using web technologies (HTML/CSS/JavaScript) for the UI and Rust for the native backend. Version 2 is used in this project.
TypeScript	A statically typed superset of JavaScript that compiles to plain JavaScript. Used for the entire MechanicPro frontend.
WebView	An embedded browser component that renders web content inside a native application window. Tauri uses the operating system's native WebView (Chromium/WebView2 on Windows, WebKit on macOS/Linux).
Zustand	A lightweight, hook-based state management library for React. Stores are plain JavaScript objects with actions; no reducers or action creators are required.

B References

- Tauri Documentation: <https://tauri.app/>
- Tauri SQL Plugin: <https://github.com/tauri-apps/tauri-plugin-sql>
- React Documentation: <https://react.dev/>
- React Router Documentation: <https://reactrouter.com/>
- Zustand Documentation: <https://zustand.docs.pmnd.rs/>
- SQLite Documentation: <https://www.sqlite.org/docs.html>
- Vite Documentation: <https://vite.dev/>
- recharts Documentation: <https://recharts.org/>
- jsPDF Documentation: <https://artskydj.github.io/jsPDF/>